

Spring Security: Authentication vs Authorization, JWT, OAuth2 & Full Flow

Spring Security is fundamentally filter-based in servlet applications, with `SecurityFilterChain` controlling how requests are secured. The current Spring Security documentation uses `AuthorizationManager` for authorization decisions. ([Home](#))

1. Authentication vs Authorization

This is the first distinction you should memorize.

Authentication

Who are you?

Example:

```
username = avnish  
password = *****
```

The system verifies that the credentials actually belong to Avnish.

```
Authentication  
↓  
Identity established  
↓  
"User is Avnish"
```

Authorization

What are you allowed to do?

After authentication:

```
User = Avnish  
Authorities = ROLE_USER, READ_PAYMENT
```

Now:

```
GET /payments/123
```

may be allowed, while:

```
DELETE /users/123
```

may be denied.

So:

```
Authentication → WHO  
Authorization → WHAT CAN THEY DO
```

2. Principal, Credentials and Authorities

Spring Security's `Authentication` object can represent both the authentication attempt and the currently authenticated identity.

It contains concepts such as:

```
principal  
credentials  
authorities
```

For example:

```
principal    = Avnish  
credentials  = password/token during authentication  
authorities  = ROLE_USER, PAYMENT_READ
```

After successful authentication, the resulting `Authentication` is associated with the `SecurityContext`, which Spring Security stores in `SecurityContextHolder`. ([Home](#))

3. Authentication Flow Inside Spring Security

A simplified architecture is:

```
HTTP Request  
  |  
  ↓  
Servlet Filter Chain  
  |  
  ↓  
Spring Security Filters  
  |  
  ↓  
Authentication  
  |  
  ↓  
AuthenticationManager  
  |  
  ↓  
AuthenticationProvider  
  |  
  ↓  
User/token validation  
  |  
  ↓  
Authentication  
  |  
  ↓
```

```

SecurityContextHolder
  |
  ↓
AuthorizationManager
  |
  ↓
Controller

```

The important classes:

```

SecurityFilterChain
Authentication
AuthenticationManager
ProviderManager
AuthenticationProvider
SecurityContext
SecurityContextHolder
AuthorizationManager

```

Spring's common `AuthenticationManager` implementation is `ProviderManager`, which delegates authentication to one or more `AuthenticationProvider` s. ([Home](#))

4. SecurityFilterChain

Modern Spring Security configuration generally looks like this:

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .requestMatchers("/admin/**")
                    .hasRole("ADMIN")
                .anyRequest()
                    .authenticated()
            );

        return http.build();
    }
}

```

The important point:

```

Request
  ↓
SecurityFilterChain
  ↓
Security filters execute

```

Do not describe modern Spring Security as primarily based on the old `WebSecurityConfigurerAdapter` style in an interview.

5. AuthenticationManager

`AuthenticationManager` is the main authentication API.

Conceptually:

```
Authentication authenticate(Authentication authentication);
```

You give it an authentication request:

```
username/password
```

or:

```
Bearer JWT
```

and it attempts to authenticate it.

The common implementation is:

```
AuthenticationManager
  ↓
ProviderManager
  ↓
AuthenticationProvider(s)
```

Each provider knows how to authenticate a particular type of credential. For example, `DaoAuthenticationProvider` handles username/password, while `JwtAuthenticationProvider` handles JWT bearer authentication. ([Home](#))

6. AuthenticationProvider

Think of:

```
AuthenticationProvider = specialist
```

Examples:

```
DaoAuthenticationProvider
  → username/password

JwtAuthenticationProvider
  → JWT

other providers
  → other authentication mechanisms
```

`ProviderManager` can contain multiple providers and delegate to the one capable of handling the particular authentication request. ([Home](#))

7. Username + Password Flow

Suppose the client sends:

```
POST /login
Content-Type: application/json

{
  "username": "avnish",
  "password": "secret"
}
```

Conceptually:

```
Client
  ↓
/login
  ↓
Spring Security authentication mechanism
  ↓
AuthenticationManager
  ↓
DaoAuthenticationProvider
  ↓
UserDetailsService
  ↓
Database
  ↓
UserDetails
  ↓
PasswordEncoder
  ↓
password matches?
```

If successful:

```
Authentication
  ↓
SecurityContext
```

Spring Security supports `UserDetailsService` backed by stores such as JDBC or custom data stores, and password validation through `PasswordEncoder` . ([Home](#))

8. Passwords Are Not Stored as Plain Text

A common implementation is:

```
@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
```

Then:

```
passwordEncoder.matches(
    rawPassword,
    storedPassword
);
```

The database should contain an encoded password, not:

```
mypassword123
```

Spring Security's `DelegatingPasswordEncoder` supports modern encodings and an encoding identifier such as `{bcrypt}` in the stored representation. ([Home](#))

9. Where Does JWT Come In?

After username/password authentication succeeds, an application **can issue a JWT**.

For example:

```
POST /auth/login
  |
  ↓
Validate username/password
  |
  ↓
Generate access token
  |
  ↓
Return JWT
```

Response:

```
{
  "accessToken": "eyJhbGciOi..."
}
```

Then the client sends:

```
GET /payments
Authorization: Bearer eyJhbGciOi...
```

The server can authenticate the request using that token instead of asking for the password again.

10. What Is JWT?

JWT = **J**SON **W**eb **T**oken.

A JWT commonly has:

```
HEADER.PAYLOAD.SIGNATURE
```

Example structure:

```
xxxxx.yyyyy.zzzzz
```

Header

Contains metadata such as:

```
{
  "alg": "RS256",
```

```
"typ": "JWT"
}
```

Payload

Contains claims:

```
{
  "sub": "avnish",
  "iss": "https://auth.example.com",
  "aud": "payment-api",
  "scope": "payment.read",
  "exp": 1780000000
}
```

Signature

Allows the recipient to verify that the signed token hasn't been modified and, for asymmetric signing, that it was signed by the holder of the private key.

Important:

A signed JWT is not automatically encrypted.

Its payload can generally be decoded by anyone holding the token. Do not put passwords or sensitive secrets in normal JWT claims.

11. Important JWT Claims

Know these:

Claim	Meaning
<code>iss</code>	Issuer
<code>sub</code>	Subject / identity
<code>aud</code>	Intended audience
<code>exp</code>	Expiration
<code>iat</code>	Issued-at
<code>nbf</code>	Not valid before
<code>scope</code>	OAuth scopes, commonly
<code>jti</code>	Token identifier

A Spring Security resource server validates things such as the token signature and relevant claims such as `exp`, `nbf`, and `iss`; scopes can be mapped to authorities. ([Home](#))

12. JWT Is Not OAuth2

This is an extremely important interview distinction.

JWT → token format

OAuth2 → authorization framework/protocol

OAuth2 does not require JWT.

An OAuth2 access token can be:

JWT

or:

Opaque token

Spring Security supports both bearer-token approaches: JWT with `JwtDecoder`, and opaque tokens with `OpaqueTokenIntrospector`. ([Home](#))

So:

JWT and OAuth2 are not competing technologies.

They solve different problems.

13. OAuth2

OAuth2 is primarily about:

Delegated authorization

Example:

```
User
↓
Frontend Application
↓
Authorization Server
↓
Access Token
↓
API / Resource Server
```

A user can authorize an application to access permitted resources without giving that application the user's password.

Important OAuth2 actors:

Resource Owner
Client
Authorization Server
Resource Server

Example:

User → Resource Owner
Angular app → Client
Identity provider → Authorization Server
Spring Boot API → Resource Server

14. OAuth2 vs OpenID Connect

Another common interview trap.

OAuth2

Concerned primarily with:

Authorization

OpenID Connect (OIDC)

Adds:

Authentication / identity

on top of OAuth2.

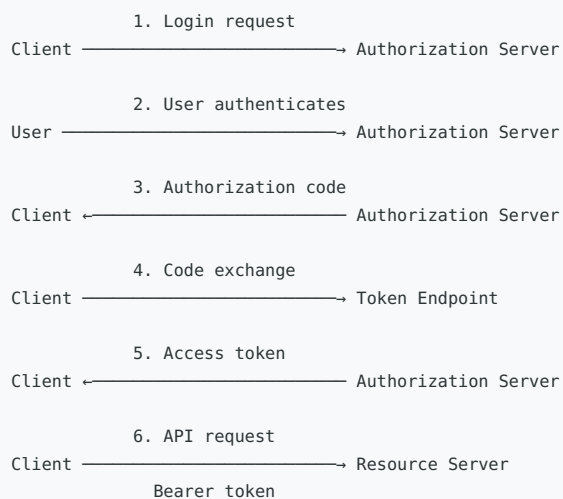
So when you hear:

"Login with Google"

that is generally an **OIDC-based authentication flow**, not just plain OAuth2 authorization. Spring Security's OAuth2 Login uses the Authorization Code flow and supports OIDC providers such as Google. ([Home](#))

15. OAuth2 Authorization Code Flow

For a web application, the conceptual flow is:



The authorization server handles the user's authentication and consent. The client then uses the resulting token to access the resource server.

Spring Security's OAuth2 Login implementation uses the Authorization Code Grant. ([Home](#))

For modern browser/public-client flows, **PKCE** is an important additional protection to understand.

16. Full JWT Resource Server Flow

This is probably the most important Spring Security flow for a backend interview.

Suppose:

```
Angular / Client
  ↓
Spring Boot API
  ↓
JWT authentication
```

Client sends:

```
GET /payments/123
Authorization: Bearer <JWT>
```

Step 1 — Request enters application

```
HTTP Request
  ↓
Servlet container
  ↓
Spring Security filter chain
```

Spring Security processes requests through servlet filters. ([Home](#))

Step 2 — Bearer token extracted

Spring Security's:

```
BearerTokenAuthenticationFilter
```

extracts the bearer token and creates a `BearerTokenAuthenticationToken`. ([Home](#))

Conceptually:

```
Authorization header
  ↓
BearerTokenAuthenticationFilter
  ↓
BearerTokenAuthenticationToken
```

Step 3 — AuthenticationManager

The authentication token is passed through the authentication machinery.

```
BearerTokenAuthenticationToken
    ↓
AuthenticationManager
    ↓
JwtAuthenticationProvider
```

Spring Security's documented resource-server flow uses the `AuthenticationManager` to authenticate the bearer token. ([Home](#))

Step 4 — JWT Decoder validates token

`JwtAuthenticationProvider` uses a JWT decoder.

```
JWT
    ↓
JwtDecoder
    ↓
Signature validation
    ↓
Claims validation
    ↓
Jwt object
```

Depending on configuration, Spring can obtain signing keys through the authorization server's JWK Set endpoint.

Typical validations include:

```
signature
expiration
not-before
issuer
```

and potentially other configured validators such as audience. ([Home](#))

17. What Does Spring Do With the JWT?

After successful validation:

```
JWT
    ↓
JwtAuthenticationProvider
    ↓
Authentication
    ↓
SecurityContextHolder
```

The authenticated principal becomes available through Spring Security.

For a JWT resource server, Spring Security can represent the principal as a `Jwt` object, and `Authentication#getName()` maps to the JWT's `sub` claim by default. ([Home](#))

You can access it:

```
@GetMapping("/profile")
public String profile(Authentication authentication) {
```

```
        return authentication.getName();
    }
}
```

or:

```
@GetMapping("/profile")
public String profile(
    @AuthenticationPrincipal Jwt jwt) {

    return jwt.getSubject();
}
```

18. Authorities From JWT

Suppose token contains:

```
{
  "scope": "payment.read payment.write"
}
```

Spring Security can map scopes to authorities such as:

```
SCOPE_payment.read
SCOPE_payment.write
```

Then:

```
.requestMatchers("/payments")
.hasAuthority("SCOPE_payment.read")
```

can protect an endpoint. Spring Security documents this scope-to-`SCOPE_` authority mapping for JWT resource servers. ([Home](#))

19. Then Authorization Happens

This is where Authentication vs Authorization becomes very clear.

Authentication has already established:

```
User = Avnish
Authorities =
  SCOPE_payment.read
  SCOPE_payment.write
```

Then Spring asks:

```
Is this authentication allowed to access /payments?
```

The authorization machinery evaluates the configured rule.

For example:

```

http.authorizeHttpRequests(auth -> auth
    .requestMatchers(HttpMethod.GET, "/payments/**")
    .hasAuthority("SCOPE_payment.read")
    .requestMatchers(HttpMethod.POST, "/payments/**")
    .hasAuthority("SCOPE_payment.write")
    .anyRequest()
    .authenticated()
);

```

Spring Security's current authorization model uses `AuthorizationManager` implementations to make these access decisions. ([Home](#))

20. Successful Request

Full sequence:

```

Client
|
| Authorization: Bearer JWT
↓
Spring Security Filter Chain
|
|
↓
BearerTokenAuthenticationFilter
|
|
↓
AuthenticationManager
|
|
↓
JwtAuthenticationProvider
|
|
↓
JwtDecoder
|
|
↓
Validate signature + claims
|
|
↓
Authentication created
|
|
↓
SecurityContextHolder
|
|
↓
AuthorizationManager
|
|
↓
ACCESS GRANTED
|
|
↓
Controller
|
|
↓
Service
|
|
↓
Response

```

This is the diagram I'd expect you to draw on a whiteboard.

21. Authentication Failure vs Authorization Failure

Very important.

401 Unauthorized

Usually means:

The request does not have valid authentication credentials.

Examples:

```
No token
Invalid token
Expired token
Bad signature
```

Flow:

```
Authentication fails
  ↓
AuthenticationEntryPoint
  ↓
401
```

Spring Security's bearer-token flow invokes an `AuthenticationEntryPoint` when authentication fails. ([Home](#))

403 Forbidden

Authentication succeeded, but:

User does not have required authority

Example:

```
User:
ROLE_USER

Endpoint requires:
ROLE_ADMIN
```

Result:

403 Forbidden

Mental model:

```
401 → "I don't know/accept who you are."

403 → "I know who you are, but you're not allowed."
```

22. Roles vs Authorities

Suppose:

```
.hasRole("ADMIN")
```

Spring Security conceptually checks an authority with the role prefix, typically:

```
ROLE_ADMIN
```

Whereas:

```
.hasAuthority("PAYMENT_READ")
```

checks that exact authority.

The current authorization DSL documents `hasRole` as a shortcut around an authority check with the configured role prefix. ([Home](#))

So:

```
ROLE_ADMIN  
PAYMENT_READ  
SCOPE_payment.read
```

are all examples of authorities represented in the `Authentication`.

23. JWT Configuration in Spring Boot

For a resource server, configuration can be relatively small.

For example:

```
@Bean  
SecurityFilterChain securityFilterChain(HttpSecurity http)  
    throws Exception {  
  
    http  
        .authorizeHttpRequests(auth -> auth  
            .requestMatchers("/public/**").permitAll()  
            .anyRequest().authenticated()  
        )  
        .oauth2ResourceServer(oauth2 -> oauth2  
            .jwt(Customizer.withDefaults())  
        );  
  
    return http.build();  
}
```

And configuration may point to an issuer:

```
spring:  
  security:  
    oauth2:  
      resourceserver:  
        jwt:  
          issuer-uri: https://auth.example.com
```

Spring Security's resource-server support is specifically designed to validate OAuth2 bearer tokens and JWTs. ([Home](#))

24. What Happens With `issuer-uri`?

Conceptually:

```
Spring Boot starts
↓
Find issuer metadata
↓
Discover authorization server configuration
↓
Discover JWK Set URI
↓
Obtain public signing keys
↓
Validate incoming JWTs
```

At request time:

```
JWT
↓
find matching key
↓
verify signature
↓
validate claims
↓
create Authentication
```

The important security principle:

The resource server does not trust a JWT simply because it is syntactically valid.

It validates its signature and claims.

25. How JWT Signature Verification Works

Imagine the authorization server uses:

Private Key

to sign the token.

The resource server has access to:

Public Key

Conceptually:

Authorization Server

```
JWT claims
+
Private Key
↓
Signature

Resource Server

JWT
+
Public Key
↓
Verify signature
```

This is one reason asymmetric algorithms such as RSA are common in distributed systems.

You don't need to give every microservice the private signing key.

26. Access Token vs Refresh Token

Access token

Used to call APIs.

```
Authorization: Bearer <access-token>
```

Usually short-lived.

Refresh token

Used to obtain a new access token without requiring the user to authenticate again.

Conceptually:

```
Access token expires
↓
Refresh token
↓
Authorization Server
↓
New access token
```

A refresh token is generally handled differently from an API bearer access token and should be protected carefully.

27. Stateful Session vs Stateless JWT

Session-based

```
Login
↓
Server creates session
↓
Session ID
```

```
↓
Client sends session ID
↓
Server looks up session
```

Server stores authentication state.

JWT-based API

```
Login/token acquisition
↓
JWT
↓
Client sends JWT on every request
↓
Resource server validates JWT
```

The API doesn't need to maintain a server-side login session for each request.

This is one reason JWT bearer authentication is attractive in distributed systems.

28. But JWT Does Not Mean “No State Anywhere”

This is an interview trap.

Even if your API is stateless:

```
Authorization Server
```

may maintain:

- users
- client registrations
- refresh tokens
- consent
- key material
- revocation information

“Stateless JWT API” generally means the **resource server doesn't need a server-side session to authenticate each request.**

29. JWT vs Session

Session	JWT
Server stores session state	Token carries claims
Client usually sends session ID	Client sends bearer token
Session lookup may be required	Token can be locally validated
Easy immediate server-side invalidation	Revocation requires additional design
Common browser pattern	Common API/microservice pattern

Don't claim JWT is universally better. It changes the trade-offs.

30. JWT Revocation Problem

Suppose:

JWT expires in 30 minutes

At 5 minutes:

User account disabled

The JWT could still be cryptographically valid until expiry.

This is one of the trade-offs of self-contained tokens.

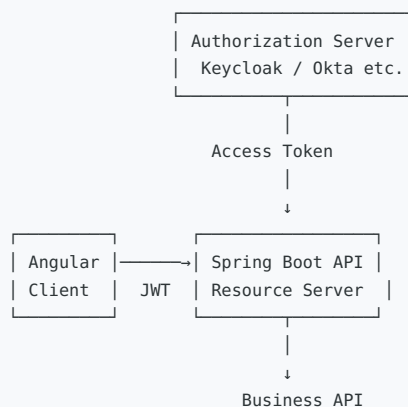
Possible approaches include:

short access-token lifetime
refresh-token rotation
token revocation/introspection
deny lists
versioned user/session state

The appropriate choice depends on the architecture and security requirements.

31. OAuth2 Full Architecture

A realistic architecture can look like:



The flow:

1. User authenticates with Authorization Server
2. Authorization Server issues tokens
3. Client calls Spring API
4. API validates access token
5. Spring creates Authentication
6. AuthorizationManager checks authorities
7. Controller executes

32. OAuth2 Login vs Resource Server

These are often confused.

OAuth2 Login

Your application itself lets a user log in through an external provider.

Example:

```
"Login with Google"
```

Spring Security's OAuth2 Login uses the authorization-code flow. ([Home](#))

OAuth2 Resource Server

Your application is an API that receives an access token.

```
Client
↓
Bearer JWT
↓
Spring Boot API
```

Your backend validates that token.

So:

```
OAuth2 Login
→ application is an OAuth client

OAuth2 Resource Server
→ application protects APIs
```

33. JWT Custom Authentication vs OAuth2

Suppose **you build your own** `/login` endpoint:

```
POST /auth/login
↓
Validate DB username/password
↓
Generate JWT
```

That's a custom authentication/token-issuance architecture.

It does **not automatically mean you've implemented OAuth2**.

OAuth2 involves defined protocol roles and flows around an authorization server, client, resource server, authorization grants, tokens, etc.

34. CSRF and JWT

This is another interview trap.

You may hear:

"JWT means disable CSRF."

That's too simplistic.

The question is:

How is the credential transported?

If the browser automatically sends an authentication cookie:

Cookie: SESSIONID=...

CSRF can matter because the browser automatically attaches cookies.

If an SPA explicitly puts a bearer token into:

Authorization: Bearer <token>

the browser does not automatically attach that header to arbitrary cross-site requests in the same way it does cookies.

So CSRF handling depends on the authentication architecture. Spring Security provides CSRF protection through `CsrfFilter`; it should not be disabled blindly. ([Home](#))

35. CORS vs CSRF

You've already studied CORS, so connect them:

```
CORS
→ Can browser JavaScript call this origin?

CSRF
→ Can another site induce an authenticated request?

Authentication
→ Who is making the request?

Authorization
→ Is this identity allowed?
```

Different problems.

36. Method-Level Authorization

Security doesn't have to be only at URLs.

You can use method security, for example:

```
@PreAuthorize("hasRole('ADMIN')")
public void deletePayment(Long id) {
    ...
}
```

Conceptually:

```
HTTP authorization
+
method authorization
```

Spring Security's authorization architecture supports both request-based and method-based authorization through `AuthorizationManager`-based infrastructure. ([Home](#))

37. Full Flow: Custom Login + JWT

This is worth memorizing because interviewers often ask for the entire flow.

```

LOGIN

Client
|
| username + password
↓
POST /auth/login
|
↓
Authentication Filter / Controller
|
↓
AuthenticationManager
|
↓
DaoAuthenticationProvider
|
↓
UserDetailsService
|
↓
Database
|
↓
PasswordEncoder
|
↓
Authentication SUCCESS
|
↓
Generate JWT
|
↓
Return JWT

```

Then later:

```

PROTECTED API

Client
|
| Authorization: Bearer JWT
↓

```

```

SecurityFilterChain
|
↓
BearerTokenAuthenticationFilter
|
↓
AuthenticationManager
|
↓
JwtAuthenticationProvider
|
↓
JwtDecoder
|
├─ verify signature
├─ validate expiration
├─ validate issuer/audience as configured
└─ map authorities
|
↓
SecurityContextHolder
|
↓
AuthorizationManager
|
↓
Controller

```

That's the end-to-end answer.

38. Full Flow: OAuth2 + JWT

Now change the architecture:

```

LOGIN

User
|
↓
Angular
|
↓
Authorization Server
|
├─ Authenticate user
├─ Consent
|
↓
Authorization Code
|
↓
Client / backend
|
↓
Token Endpoint
|
↓
Access Token
|
↓
Angular

```

Then:

API CALL

```
Angular
|
| Authorization: Bearer JWT
↓
Spring Boot Resource Server
|
↓
BearerTokenAuthenticationFilter
|
↓
JwtAuthenticationProvider
|
↓
JwtDecoder
|
↓
Authentication
|
↓
AuthorizationManager
|
↓
Controller
```

39. Most Important Classes to Know

For a backend interview, know these:

```
SecurityFilterChain
HttpSecurity

SecurityContext
SecurityContextHolder

Authentication
GrantedAuthority

AuthenticationManager
ProviderManager
AuthenticationProvider

UserDetails
UserDetailsService
PasswordEncoder
DaoAuthenticationProvider

BearerTokenAuthenticationFilter
JwtAuthenticationProvider
JwtDecoder

AuthorizationManager
```

And for OAuth2:

```
OAuth2AuthorizationRequest
OAuth2AuthorizedClient
OAuth2LoginAuthenticationFilter
OAuth2AuthenticationToken
```

You don't need to memorize every Spring Security class. Understand **responsibility and flow**.

40. Common Interview Questions

Fundamentals

1. Authentication vs authorization?
2. What is `SecurityContextHolder` ?
3. What is `Authentication` ?
4. What are `GrantedAuthority` s?
5. What is `AuthenticationManager` ?
6. What is `AuthenticationProvider` ?
7. What is `ProviderManager` ?

JWT

8. What is JWT?
9. Explain JWT header, payload and signature.
10. Is JWT encrypted?
11. What is the difference between signing and encryption?
12. What are `iss` , `sub` , `aud` , `exp` ?
13. How does Spring validate a JWT?
14. How are JWT scopes converted into authorities?
15. What happens when a JWT expires?
16. How would you revoke a JWT?
17. Access token vs refresh token?

OAuth2

18. What problem does OAuth2 solve?
19. OAuth2 vs JWT?
20. OAuth2 vs OIDC?
21. Explain Authorization Code flow.
22. What is an Authorization Server?
23. What is a Resource Server?
24. What is a Client?
25. Why use PKCE?

Spring Security

26. Explain the `SecurityFilterChain` .
27. Explain JWT authentication end-to-end.
28. What is `BearerTokenAuthenticationFilter` ?
29. What is `JwtAuthenticationProvider` ?
30. Authentication failure vs authorization failure?
31. 401 vs 403?
32. `hasRole()` vs `hasAuthority()` ?
33. How do you implement method-level security?
34. Why use `PasswordEncoder` ?

35. Why shouldn't passwords be stored directly?

41. Three Questions That Test Real Understanding

Question 1

A request has a valid JWT but the user doesn't have `PAYMENT_WRITE` . What happens?

Correct reasoning:

```
JWT valid
↓
Authentication succeeds
↓
User identity established
↓
Required authority missing
↓
Authorization fails
↓
403
```

Question 2

A request contains an expired JWT. Which stage fails?

```
Bearer token extracted
↓
JWT authentication
↓
JWT validation
↓
exp validation fails
↓
Authentication failure
↓
401
```

Question 3

Does Spring Security call the database for every JWT request?

Not necessarily.

With a properly configured JWT resource server:

```
Request
↓
JWT
```

```
↓
signature/claim validation
↓
Authentication
```

The resource server can validate the token locally using the appropriate signing key rather than querying the user database for every request. That's a major property of self-contained JWT access tokens. Spring Security's JWT resource-server support is specifically designed around local JWT decoding/validation. ([Home](#))

42. Interview Cheat Sheet

Memorize this:

```
Authentication
↓
Who are you?

Authorization
↓
What can you do?

JWT
↓
Token format

OAuth2
↓
Authorization framework

OIDC
↓
Authentication/identity layer on OAuth2

SecurityFilterChain
↓
Security filters for the request

AuthenticationManager
↓
Coordinates authentication

AuthenticationProvider
↓
Actually knows how to authenticate

SecurityContextHolder
↓
Stores current Authentication

GrantedAuthority
↓
Roles/scopes/permissions

AuthorizationManager
↓
Makes access decision

401
↓
Authentication problem

403
↓
```

The single flow to draw in an interview

```

Client
|
| Authorization: Bearer JWT
↓
SecurityFilterChain
↓
BearerTokenAuthenticationFilter
↓
AuthenticationManager
↓
JwtAuthenticationProvider
↓
JwtDecoder
↓
Validate JWT
↓
Authentication
↓
SecurityContextHolder
↓
AuthorizationManager
↓
Controller

```

That flow, plus **OAuth2 vs JWT vs OIDC**, covers a large portion of the Spring Security questions expected at your experience level.

BPS-Specific Questions — Inspect Your Codebase

Don't assume these are the answers; inspect your BPS implementation and identify the actual mechanism.

Authentication

- Does BPS implement its own username/password login, or does an external Identity Provider authenticate users?
- Is JWT generated by BPS or by another authentication/authorization service?
- Do you have a custom `UserDetailsService`, `AuthenticationProvider`, or `AuthenticationManager`?
- Where is authentication established in the request flow?

JWT

- Does BPS receive:

```
Authorization: Bearer <token>
```

- Is the token actually a JWT?
- Which claims does BPS use?
- Does BPS validate `iss`, `aud`, `exp`, signature, or other claims?
- Are roles represented as `ROLE_*`, scopes as `SCOPE_*`, or some custom authorities?
- Does BPS use a shared public key/JWK endpoint?

Authorization

- Which BPS endpoints are `permitAll()` ?
- Which require authentication?
- Are any endpoints protected with `hasRole()` or `hasAuthority()` ?
- Is there method-level authorization such as `@PreAuthorize` ?

Full-flow inspection

Trace **one real BPS API request** from:

```
Controller
↓
SecurityFilterChain
↓
Authentication filter
↓
AuthenticationManager/Provider
↓
SecurityContext
↓
Authorization
↓
Controller
```

and identify the actual classes from the codebase. That will turn this from a theoretical interview topic into something you can explain from your own production experience.